A CASE-STUDY BASED REPORT

ON

Hybrid Network Intrusion Detection Using Random Forest and Deep Neural Networks

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

R.C. Sai Sharan roll 2520030128

T. Abhinav roll 2520030075

Under the Esteemed guidance of

Jyothsna Datti

Asst.Professor, Dept. of CSE



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract **3**

Introduction **4**

Background 4
Motivation for the Project 4
Overview of the Work Carried Out 4
Scope and Responsible-Use Statement 5

Problem Statement **6**

Case-Study Scenario 6
Central Case-Study Question 6
Formal Problem Definition 6
Objectives that Guide the Study 6
Assumptions and Limitations 6

Dataset and Exploratory Data Analysis **8**

About the Dataset 8
Purpose of EDA 8
Preparation Steps Actually Performed 8
EDA Exploration - Implementation 9
Summary of EDA Findings 10

Algorithm Selection and Implementation **12**

Conclusion **15**

Abstract

These days cyber attacks like DoS, Probe, R2L and U2R are increasing a lot. Normal security tools mostly work on fixed rules, so they often miss new or tricky attacks. In this case study we made a Network Intrusion Detection System (NIDS) which uses two machine learning models, Random Forest and a Deep Neural Network (DNN), and also combines both of them into a hybrid model. The system tells whether a network connection is normal or which type of attack it is.

We used a dataset of 2,000 connection records in the NSL-KDD format, with 400 records for each of the five classes (normal, DoS, Probe, R2L, U2R). We first checked the data, created some new features such as traffic volume and login risk, scaled the numbers and encoded the text columns. Then the data was split 80% for training (1,600 records) and 20% for testing (400 records).

On the test data, Random Forest gave **96.50%** accuracy, the DNN gave **95.25%** and the hybrid model gave **96.25%**. Normal traffic was predicted correctly every time. Most mistakes were between R2L, U2R and Probe attacks. We also made a threat score and a security alert, and a small Flask web page where a CSV file can be uploaded to get the predictions.

Keywords: Network Intrusion Detection, Random Forest, Deep Neural Network, Hybrid Model, NSL-KDD, Machine Learning.

Introduction

Background

Today almost everything like banking, hospitals, colleges and companies depends on computer networks. Because of this, attackers also try to enter these networks to steal data or stop the services. A network has a huge amount of traffic every second, so it is not possible for a person to watch all of it. Old security tools can only catch the attacks that they already know.

Machine learning can help here. If we give the model many examples of normal and attack traffic, it can learn the pattern and then predict for new traffic by itself. In this project we classify traffic into five classes:

- **Normal** – safe, legitimate traffic.
- **DoS (Denial of Service)** – the attacker floods a server so that real users cannot use it.
- **Probe** – scanning the network to find open ports or weak points.
- **R2L (Remote to Local)** – an outside person tries to get access to a local machine, for example by guessing passwords.
- **U2R (User to Root)** – a normal user tries to get admin (root) rights.

Before starting we studied 25 research papers on intrusion detection. Most of them use Random Forest, SVM, Decision Tree, DNN, CNN, LSTM or hybrid models. We found that many papers use only one type of model, many use the old KDD99 dataset, deep learning models need lots of computing power, and class imbalance is also a problem. So we decided to compare a Random Forest with a DNN and also combine them.

Motivation for the Project

Attacks like DoS, Probe, R2L and U2R can cause data loss and stop services, and traditional methods may not find new attack patterns. So we wanted to build a system that can automatically find malicious traffic with good accuracy and fewer false alarms. We also thought that only giving a prediction is not enough, so our system also shows a threat level and how confident the models are.

Overview of the Work Carried Out

1. Loaded the dataset (2,000 rows, 44 columns) and checked missing values and class balance.
2. Created 8 new features and a threat behavior score, and divided the records into LOW, MEDIUM and HIGH threat levels.
3. Separated input and target, and made a preprocessing pipeline (scaling and one-hot encoding).
4. Split the data 80:20 and trained Random Forest and DNN models.
5. Checked both models using accuracy, precision, recall, F1-score and confusion matrix.

6. Made a hybrid model by averaging the probabilities of both models and added a security alert system.
7. Made a Flask web page to upload a CSV file and see the results.

Scope and Responsible-Use Statement

This project only does classification of already extracted network connection records into the five classes. Capturing live packets and blocking traffic automatically are not part of this work.

The system should be used only for defence, that is, for checking networks which we own or have permission to monitor. It is only a help for the security person and the alerts should be checked by a human before taking any action. The dataset does not have any personal data.

Problem Statement

Case-Study Scenario

Imagine a company whose servers receive thousands of connections every minute. Most of them are normal, but some are attacks, such as someone flooding the web server, scanning the open ports, trying many passwords, or a user trying to become admin. The security team cannot check every connection by hand and the old rule-based filter misses unknown attacks. They need a system which reads each connection and tells whether it is normal or an attack, and how serious it is.

Central Case-Study Question

Can Random Forest, a Deep Neural Network, or a hybrid of both correctly classify network traffic as normal or as one of the four attack types, and can we use their output to give a reliable security alert with fewer false alarms?

Formal Problem Definition

The input is a network connection described by its features (the 41 original attributes plus our new features). The output is its class: Normal, DoS, Probe, R2L or U2R. We have to train a model on labelled data so that it predicts the class of new, unseen connections with high accuracy. We train Random Forest and DNN separately, and the hybrid model takes the average of their predicted probabilities and selects the class with the highest value. Performance is checked with accuracy, precision, recall and F1-score on the test data.

Objectives that Guide the Study

- To classify network traffic as normal or attack.
- To identify the different attack types (DoS, Probe, R2L, U2R).
- To compare Random Forest and DNN on the same data.
- To combine both models and see if the hybrid gives better results.
- To create a threat score and threat level from the traffic behaviour.
- To generate security alerts and reduce false alarms.

Assumptions and Limitations

Assumptions

- The labels in the dataset are correct.
- The 2,000 records are enough to represent the five classes.
- One 80:20 split is fine for comparing the models.

Limitations

- The dataset is small and perfectly balanced (400 per class). Real networks have mostly normal traffic, so real results may be lower.

- We used only one train-test split and no cross-validation, so the small difference between RF and DNN may not be very reliable.
- The features are from NSL-KDD which is old, and we did not test on newer datasets like UNSW-NB15.
- The weights of the threat score were chosen by us, not learned. In our data no record reached HIGH level, so the CRITICAL alert was never triggered.
- We tested only on stored records, not on live traffic.

Dataset and Exploratory Data Analysis

About the Dataset

We used the file ML_Network_Intrusion_Dataset_Literature_Aligned.csv. It follows the NSL-KDD format, which is a popular benchmark dataset for intrusion detection. It has **2,000 records and 44 columns** and there are **no missing values**. All five classes have exactly 400 records. The columns are of these types:

- **Basic connection features:** duration, protocol_type, service, flag, src_bytes, dst_bytes etc.
- **Content features:** num_failed_logins, logged_in, root_shell, su_attempted, num_root etc.
- **Time based features:** count, srv_count, serror_rate, rerror_rate, same_srv_rate etc.
- **Host based features:** dst_host_count, dst_host_srv_count, dst_host_serror_rate etc.
- **Target and extra columns:** attack_type (target), label and difficulty_level.

Three columns are categorical (protocol_type, service, flag) and the others are numerical. We did not use label and difficulty_level as input because they come from the labelling.

Purpose of EDA

The purpose of EDA is to understand the data before building the model. We checked the size of the data, missing values, whether the classes are balanced, which columns are numerical or categorical, and whether our new features and threat score can separate normal and attack traffic.

Preparation Steps Actually Performed

1. Loaded the CSV file with pandas and printed the shape (2000, 44) and missing values (0).
2. Created 8 new features (shown in the table below).
3. Made the threat behavior score from 6 normalised features and divided it into LOW (below 30), MEDIUM (30 to 60) and HIGH (60 and above).
4. Took attack_type as the target and removed attack_type, label, difficulty_level and threat_level from the input. This gave 56 input features (53 numerical and 3 categorical).
5. Split the data into 1,600 training and 400 testing records (stratified, random_state = 42), so every class has 320 train and 80 test records.
6. Used StandardScaler for numerical columns and OneHotEncoder for categorical columns. After encoding there are 70 input features. It was fitted only on training data.

Table 1: New features created

Feature	How it is calculated
traffic_volume	src_bytes + dst_bytes
upload_download_ratio	src_bytes / (dst_bytes + 1)

Feature	How it is calculated
connection_intensity	count / (duration + 1)
service_error_score	average of the 4 error rates
service_consistency	same_srv_rate - diff_srv_rate
host_anomaly_score	average of 4 dst_host rates
privilege_activity	num_compromised + num_root + num_shells + num_file_creations
login_risk	num_failed_logins + su_attempted + root_shell

The threat score uses these weights: traffic volume 0.20, service error 0.20, host anomaly 0.15, login risk 0.15, privilege activity 0.15 and connection intensity 0.15. The result is multiplied by 100.

EDA Exploration - Implementation

First we loaded the data and checked the shape, missing values and class counts:

```
df = pd.read_csv("ML_Network_Intrusion_Dataset_Literature_Aligned.csv")
print(df.shape)                    # (2000, 44)
print(df.isnull().sum().sum())     # 0
print(df["attack_type"].value_counts())
sns.countplot(data=df, x="attack_type")
```

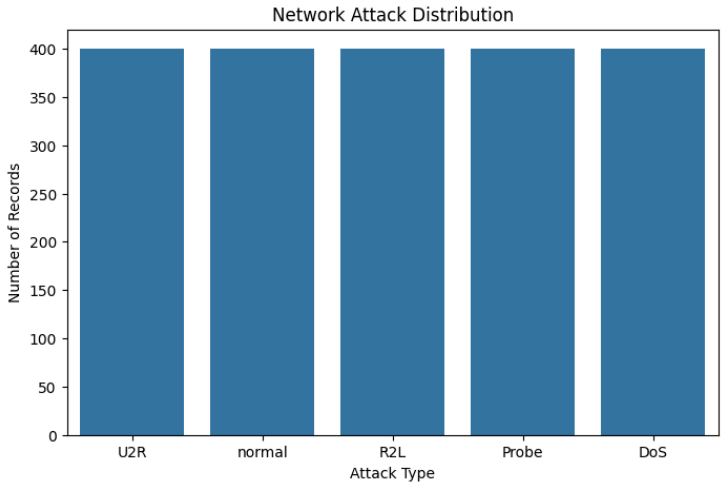


Figure 1: Distribution of attack types

Figure 1 shows that all five classes have 400 records each, so the data is balanced and we did not need any resampling.

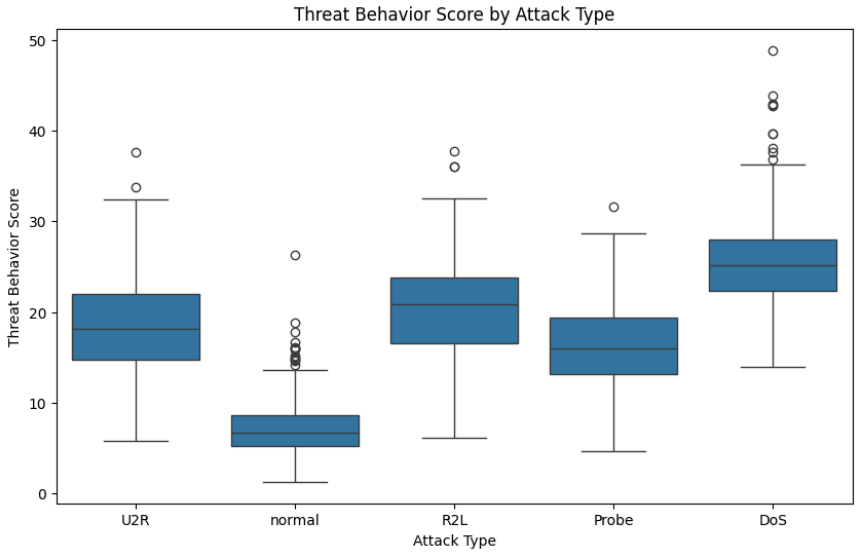


Figure 2: Threat behavior score for each attack type

In Figure 2 we can see that normal traffic has a low threat score (median about 7) and DoS has the highest (median about 25). R2L, U2R and Probe are in the middle and overlap with each other, so these classes may be difficult for the models.

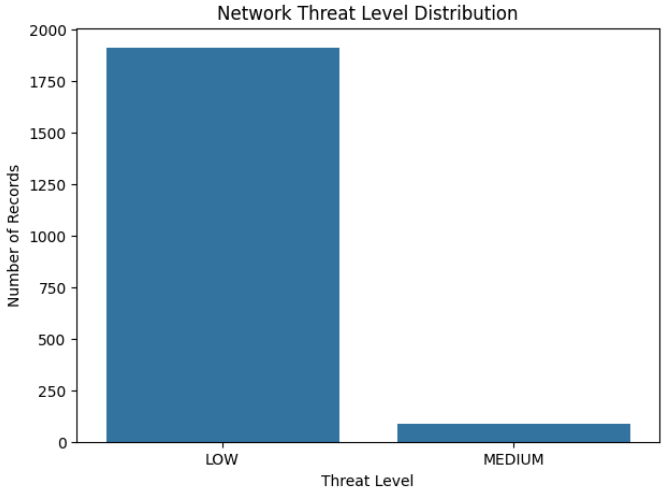


Figure 3: Threat level distribution

Figure 3 shows 1,912 records in LOW and 88 records in MEDIUM level. No record is in HIGH level because the highest score is 48.85. Most of the MEDIUM records (about 64) are DoS attacks.

Summary of EDA Findings

- The dataset has 2,000 records and 44 columns, with no missing values.
- All five classes are balanced with 400 records each.
- The columns have very different ranges (for example traffic_volume goes up to 76,276 while rates are between 0 and 1), so scaling is needed, mainly for the DNN.

- Three columns are categorical, so we used one-hot encoding.
- Normal traffic has a low threat score and DoS has a high one. R2L, U2R and Probe look similar, so most errors are expected in these classes.

Algorithm Selection and Implementation

Algorithms Used

Random Forest: It makes many decision trees and takes the final answer by voting. It works well on table type data, does not need scaling and does not overfit easily like a single decision tree. We used it as a strong base model.

Deep Neural Network (DNN): It has many layers of neurons and can learn complex non-linear patterns from the data. It needs scaled data and more training time.

Hybrid RF + DNN: We take the probability given by both models, average them (0.5 each) and pick the class with the highest value. We did this so that one model can cover the mistakes of the other.

Random Forest Implementation

```
rf_pipeline = Pipeline([
  ("preprocessor", preprocessor),
  ("classifier", RandomForestClassifier(
      n_estimators=300, max_depth=18,
      min_samples_split=4, random_state=42,
      class_weight="balanced"))])
rf_pipeline.fit(X_train, y_train)
```

Deep Neural Network Implementation

The DNN has an input layer of 70 features, then hidden layers with 128, 64 and 32 neurons (ReLU), dropout of 0.30 and 0.20 to avoid overfitting, and an output layer with 5 neurons (softmax). It has 19,589 parameters. We used Adam optimizer, batch size 32, 20% validation split and EarlyStopping with patience 8.

```
dnn = Sequential([
  Dense(128, activation="relu", input_shape=(70,)), Dropout(0.30),
  Dense(64, activation="relu"), Dropout(0.20),
  Dense(32, activation="relu"),
  Dense(5, activation="softmax")])
dnn.compile(optimizer="adam",
  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

Training stopped at epoch 18 out of 50 because of early stopping. The best validation loss was 0.1222 at epoch 10 and those weights were restored.

Hybrid Model and Security Alert

The security alert is decided by these rules, checked in this order:

1. If threat score is 70 or more, the alert is CRITICAL ALERT.
2. If RF and DNN disagree and confidence is below 70%, the alert is REVIEW REQUIRED.
3. If the hybrid prediction is not normal, the alert is INTRUSION DETECTED.

4. Otherwise it is NORMAL TRAFFIC.

Results and Evaluation

We tested all the models on the same 400 test records.

Table 2: Model performance on test data

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	96.50%	96.51%	96.50%	96.50%
Deep Neural Network	95.25%	95.30%	95.25%	95.25%
Hybrid RF + DNN	96.25%	-	-	-

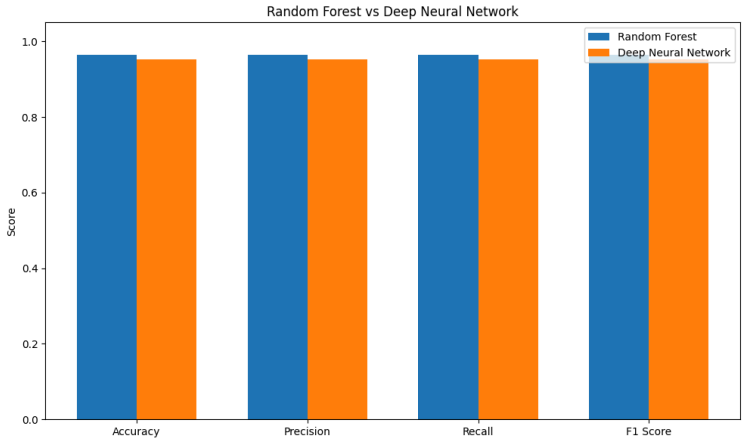


Figure 4: Random Forest vs Deep Neural Network

Random Forest is the best with 96.50% accuracy, and the hybrid is very close with 96.25%. The DNN got 95.25%. In the per-class report, normal traffic has 1.00 for all metrics in both models. DoS is also very good (F1 0.99 for RF). R2L is the weakest class (F1 0.93 for RF and 0.91 for DNN).

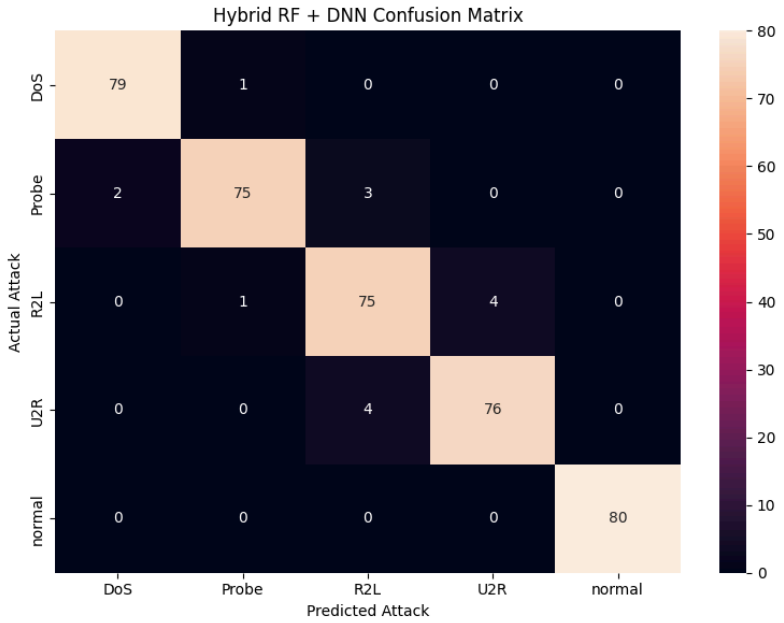


Figure 5: Confusion matrix of the hybrid model

The hybrid model predicted 385 out of 400 records correctly. All 80 normal records were correct, so there were no false alarms in the test data. The 15 mistakes are all between attack classes, for example Probe confused with DoS or R2L, and R2L confused with U2R. This is the same overlap that we saw in the threat score graph.

The RF and DNN gave the same prediction for 95.25% of test records and gave different predictions for 19 records. These disagreement cases get the REVIEW REQUIRED alert, so a person can check them instead of trusting one model blindly.

Web Dashboard

We also made a simple Flask web application. The user uploads a CSV file, the system creates the extra features, runs RF, DNN and the hybrid model, and shows the predictions, threat score, threat level, confidence and the security alert in a table. One thing we noticed is that the threat score is normalised on the uploaded file only, so for small files the scores are not same as in the full dataset. This can be fixed in future by saving the normalisation values from the training data.

Conclusion

In this case study we built a machine learning based Network Intrusion Detection System using the NSL-KDD format dataset. We trained a Random Forest and a Deep Neural Network and also combined them. Random Forest got 96.50% accuracy, DNN got 95.25% and the hybrid got 96.25% on the test data. All models detected normal traffic perfectly and the errors were only between the attack types, mostly R2L, U2R and Probe.

The hybrid model did not give better accuracy than Random Forest in our experiment, but it is still useful because the model agreement, confidence and threat score help to find uncertain cases and reduce false alarms.

Future work: testing with newer and real-world datasets like UNSW-NB15 and CSE-CIC-IDS2018, using cross-validation, making the system work on live traffic, adding automatic alerts and notifications, and improving detection of new unseen attacks.

Signature of the Team members

R.C. Sai Sharan (2520030128)

T. Abhinav (2520030075)